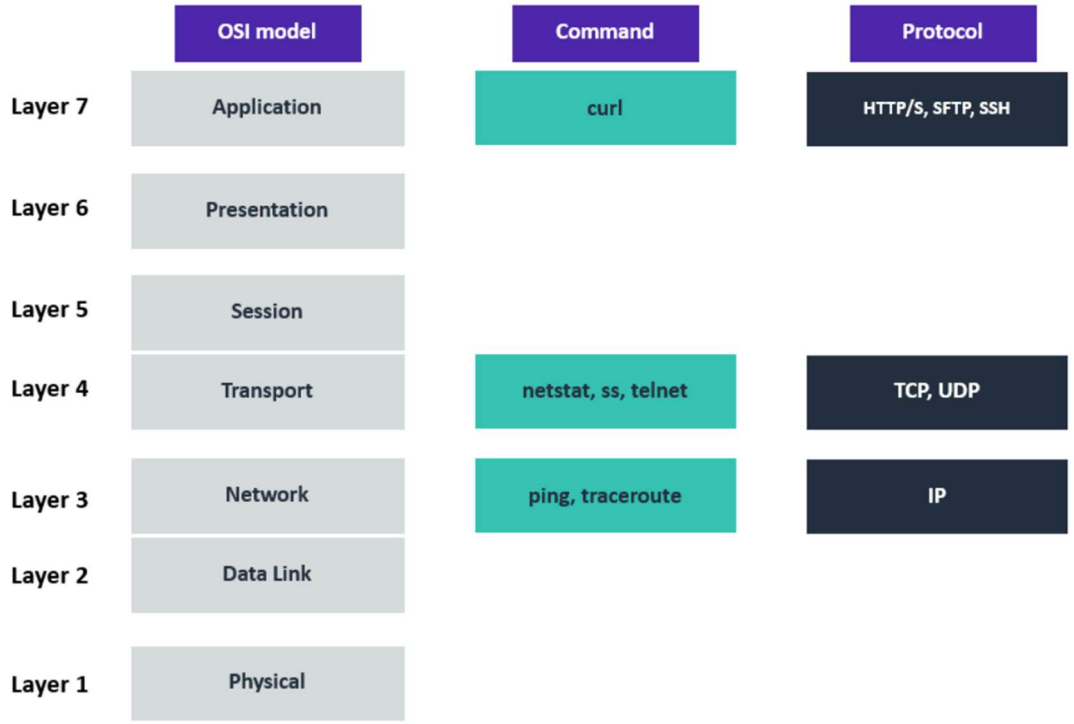
265--Lab - Comandos de solución de problemas del protocolo de Internet

En esta práctica se muestran algunos comandos de Linux para la resolución de problemas.

El modelo OSI y la resolución de problemas

En la siguiente imagen se muestra la capa a la que corresponden algunos comandos de Linux para la resolución de problemas.



Capa de red: comandos *ping* y *traceroute*

Para verificar la conectividad y asegurarse de que las peticiones ICMP son permitidas, ejecutamos el comando *ping 8.8.8.8 -c 5*

Con la opción -c especificamos el número de peticiones.

El ICMP (Protocolo de Mensajes de Control de Internet) es un protocolo de la capa de red del modelo OSI que los dispositivos de red utilizan para comunicarse problemas en la transmisión de datos.

```
ec2-user@ip-10-0-10-188:~
[ec2-user@ip-10-0-10-188 ~]$ ping 8.8.8.8 -c 5
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=5.53 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=5.56 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=5.58 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=5.57 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=5.58 ms

--- 8.8.8.8 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 5.532/5.567/5.583/0.050 ms
```

La respuesta es exitosa.

En el caso de que la conexión sea tardada o haya pérdida de paquetes ejecutamos el comando *traceroute 8.8.8.8*

El comando *traceroute se* utiliza para rastrear la ruta que sigue un paquete desde la computadora hasta un destino específico.

Si la pérdida sucede hacia el servidor, el problema probablemente es del ISP. Si la pérdida sucede hacia el recurso de AWS, se requiere investigar otros factores.

8.8.8.8 es la dirección de los servidores de Google.

```
ec2-user@ip-10-0-10-188:~
[ec2-user@ip-10-0-10-188 ~]$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1  240.1.228.71 (240.1.228.71)  6.386 ms 240.1.228.14 (240.1.228.14)  7.999 ms
240.1.228.12 (240.1.228.12)  6.183 ms
 2  242.16.83.105 (242.16.83.105)  5.928 ms 242.4.195.197 (242.4.195.197)  7.872
 ms 242.4.194.69 (242.4.194.69)  5.659 ms
 3  * * 99.82.10.6 (99.82.10.6)  7.994 ms
 4  * * 99.82.10.9 (99.82.10.9)  7.501 ms
 5  * * 192.178.105.41 (192.178.105.41)  5.599 ms
 6  142.251.55.201 (142.251.55.201)  6.026 ms dns.google (8.8.8.8)  7.396 ms  5.
628 ms
```

Cada número es un salto o hop.

En los saltos 1 y 2 se ven direcciones que suelen ser routers internos. Los tiempos de respuesta son muy bajos (alrededor de 6ms a 7ms), lo que indica una conexión local muy rápida.

En los saltos 3,4 y 5, los asteriscos (**) significan que algunos de los paquetes enviados no recibieron respuesta de vuelta (timeout) o que esos routers específicos tienen configurado el protocolo ICMP para no responder a solicitudes de diagnóstico por razones de seguridad. Sin embargo, como el rastro continúa, sabemos que el tráfico está pasando a través de ellos sin problemas.

En el salto 6 los paquetes llegan con éxito a dns.google (8.8.8.8) en solo 6 saltos. El tiempo final es de 7.396 ms, lo cual es una latencia excelente para una conexión a internet.

Tres asteriscos (***) indican un salto fallido.

La pérdida de paquetes se muestra en porcentajes.

Capa de transporte: comandos *netstat* y *telnet*

Para verificar si alguno de los puertos en una subred está comprometido ejecutamos el comando *netstat -tp* desde una instancia de la subred.

Este comando se utiliza para visualizar las conexiones de red activas y ver qué programas específicos las están originando.

La opción *-t* filtra los resultados para mostrar únicamente las conexiones que utilizan el protocolo TCP.

La opción *-p* muestra el PID del proceso y el nombre del programa que es dueño de esa conexión.

```
ec2-user@ip-10-0-10-188:~
[ec2-user@ip-10-0-10-188 ~]$ netstat -tp
(No info could be read for "-p": geteuid()=1000 but you should be root.)
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name

tcp        0     64 ip-10-0-10-188.us-w:ssh dsl-187-154-111-2:64123 ESTABLISHED -
```

El mensaje (No info could be read for "-p": geteuid()=1000 but you should be root.) indica que, como el comando no se ejecutó con *sudo*, el sistema no tiene permiso para mostrar el PID del programa que está usando la conexión.

El resultado indica que la conexión utiliza el protocolo TCP.

Local Address es la dirección de la instancia actual. El nombre termina en :ssh, lo que significa que el puerto local abierto es el 22 (el puerto estándar de SSH).

State (ESTABLISHED) significa que la conexión es activa y exitosa.

Una variante del comando anterior es *netstat -tlp*, que sirve para identificar que servicios están escuchando.

La opción -l es para mostrar únicamente los puertos que están abiertos y esperando recibir a alguien.

Otra variante es el comando *netstat -ntlp,* que permite ver que servicios están escuchando pero con una visualización más limpia y técnica.

La opción *-n* es para mostrar las direcciones IP y los puertos en forma de números brutos en vez de nombres.

El comando *Telnet* sirve para verificar la conectividad a nivel de aplicación hacia un servidor web, probando si un puerto específico está abierto y aceptando conexiones.

Instalamos telnet con sudo *yum install telnet -y*

```
[ec2-user@ip-10-0-10-188 ~]$ sudo yum install telnet -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core                                                          | 3.6 kB  00:00:00
Resolving Dependencies
--> Running transaction check
---> Package telnet.x86_64 1:0.17-65.amzn2 will be installed
--> Finished Dependency Resolution

Dependencies Resolved

=================================================================================================
 Package            Arch               Version                     Repository              Size
=================================================================================================
Installing:
 telnet             x86_64             1:0.17-65.amzn2             amzn2-core              64 k

Transaction Summary
=================================================================================================
Install  1 Package

Total download size: 64 k
Installed size: 109 k
Downloading packages:
telnet-0.17-65.amzn2.x86_64.rpm                                     |  64 kB  00:00:00
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Installing : 1:telnet-0.17-65.amzn2.x86_64                                             1/1
  Verifying  : 1:telnet-0.17-65.amzn2.x86_64                                             1/1

Installed:
  telnet.x86_64 1:0.17-65.amzn2

Complete!
```

Ejecutamos *telnet www.google.com 80*

```
ec2-user@ip-10-0-10-188:~
[ec2-user@ip-10-0-10-188 ~]$ telnet www.google.com 80
Trying 142.251.45.132...
Connected to www.google.com.
Escape character is '^]'.
```

La respuesta indica que el puerto 80 está abierto y el servidor de Google aceptó la conexión.

Esto confirma que no hay firewalls bloqueando el tráfico de salida desde la instancia hacia ese destino en ese puerto específico.

Capa de aplicación: comando *curl*

Ejecutamos el comando *curl -vLo /dev/null https://aws.com* para verificar si la petición al servidor es exitosa.

La opción -v sirve para mostrar todo el dialogo técnico en la pantalla.

La opción *-L* sirve para que, en el caso de que la dirección haya sido redireccionada, automáticamente se haga una petición a la dirección correcta.

Con la opción -o /dev/null indicamos que se descarte el contenido de la página (el código HTML).

```
ec2-user@ip-10-0-10-188:~
[ec2-user@ip-10-0-10-188 ~]$ curl -vLo /dev/null https://aws.com
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0*   Trying 3.169.1
73.19:443...
* Connected to aws.com (3.169.173.19) port 443
* ALPN: curl offers h2,http/1.1
* Cipher selection: ALL:!EXPORT:!EXPORT40:!EXPORT56:!aNULL:!LOW:!RC4:@STRENGTH
} [5 bytes data]
* TLSv1.2 (OUT), TLS handshake, Client hello (1):
} [512 bytes data]
*  CAfile: /etc/pki/tls/certs/ca-bundle.crt
*  CApath: none
```

```
{ [5 bytes data]
< HTTP/2 200
< content-type: text/html;charset=utf-8
< date: Fri, 02 Jan 2026 01:49:42 GMT
< set-cookie: aws-priv=eyJ2IjoxLCJ1dSI6MCwic3QiOjB9; Version=1; Comment="Anonymous cookie for pr
ivacy regulations"; Domain=.aws.amazon.com; Max-Age=31536000; Expires=Sat, 02 Jan 2027 01:49:42
GMT; Path=/; Secure
< set-cookie: aws_lang=en; Domain=.amazon.com; Path=/
< x-content-type-options: nosniff
< server: Server
< x-frame-options: SAMEORIGIN
```

Con el mensaje “HTTP/2 200” podemos ver que la conexión es exitosa.